OSINT Dashboard - Project Outline

A Python-based Open Source Intelligence (OSINT) aggregation and visualization dashboard. This document outlines architecture, modules, tech stack, data sources, and a phased build plan. No code included - planning document only.

1. Project Goal

Build a self-hosted dashboard that aggregates publicly available intelligence on a target (person, domain, organization, or IP address) from multiple open sources, normalizes the data, and presents it in a searchable, visual interface. This mirrors real-world OSINT tooling used in security research, threat intelligence, and investigative work.

2. Legal and Ethical Scope (define this first)

- Only query data sources that are genuinely public and permit automated access (check each source's Terms of Service before scraping). - Only investigate targets you own, have explicit permission to research, or that are clearly public entities/infrastructure (e.g., your own domains, your own social accounts, public company info). - Respect API rate limits and robots.txt where scraping is involved. - Do not build features intended to bypass authentication, paywalls, or access controls - that crosses from OSINT into unauthorized access. - Store any collected data securely; treat personal data (even if public) with care regarding retention and exposure.

3. Core Modules

3.1 Domain / Network Intelligence Module

- WHOIS lookup (registrar, creation/expiry date, registrant info if public) - DNS record enumeration (A, AAAA, MX, NS, TXT, CNAME) - Subdomain enumeration (certificate transparency logs, passive DNS sources) - SSL/TLS certificate inspection (issuer, validity, SANs) - Reverse IP lookup / ASN and hosting provider identification - Geolocation of resolved IPs

3.2 Social Media / Username Intelligence Module

- Username enumeration across public platforms (check existence of a given handle across many sites) - Public profile metadata collection (bio, join date, follower counts where publicly exposed via official APIs) - Cross-referencing a username/email against known public breach-notification aggregators (e.g., checking if an email appears in disclosed breaches, via legitimate public APIs only)

3.3 Email / Identity Intelligence Module

- Email format/validity checks - MX record and mail provider identification - Public breach exposure lookups via legitimate third-party APIs - Associated domain/company inference from email format

3.4 Image / Metadata Intelligence Module

- EXIF metadata extraction from user-supplied images (GPS coordinates, device info, timestamps if present) - Reverse image search integration (via supported public APIs)

3.5 Company / Organization Intelligence Module

- Public business registry lookups - Employee/technology footprint inference (job postings mentioning tech stack, public GitHub orgs, etc.) - News and public mention aggregation (RSS/news API search by company name)

3.6 Aggregation and Correlation Engine

- Normalizes results from all modules into a common internal data model - Cross-references findings (e.g., a domain's registrant email also appearing in the email intelligence module) - Builds a simple 'relationship graph' connecting related findings (domain -> IP -> hosting provider -> other

domains on same host)

3.7 Reporting Module

- Export findings per target as a structured report (PDF/HTML/JSON) - Timeline view of when each data point was collected - Confidence/source attribution per data point

4. Suggested Architecture

Backend: Python (FastAPI or Flask) exposing a REST API that orchestrates calls to each intelligence module. Task Queue: Celery or a simple async task runner - many OSINT lookups are slow/rate-limited and should run asynchronously rather than blocking the UI. Database: PostgreSQL or SQLite (for a single-user/home lab version) to persist collected data per investigation/case, enabling historical lookups without re-querying sources every time. Cache Layer: Redis (optional) to cache repeated lookups and respect API rate limits. Frontend: A lightweight web dashboard - options include a Python-native approach (Streamlit or Dash, fastest to build) or a separate React/Vue frontend talking to the FastAPI backend (more flexible, more work). Data Sources / APIs: A pluggable 'connector' pattern where each OSINT source (WHOIS, DNS, breach-check APIs, social platforms) is its own module implementing a common interface - makes it easy to add/remove sources later.

5. Suggested Tech Stack Summary

Layer	Suggested Tool(s)
Backend API	Python + FastAPI
Async/background jobs	Celery + Redis, or asyncio-based task queue
Database	PostgreSQL (or SQLite for local/single-user)
Frontend/Dashboard	Streamlit (fast prototype) or React (production-grade)
DNS/WHOIS libraries	Public Python libraries for WHOIS and DNS resolution
Data visualization	Plotly / matplotlib for charts; a graph library (e.g., networkx) for relationship mapping
Containerization	Docker + docker-compose for reproducible deployment on your Proxmox lab
Secrets management	Environment variables / .env file, never hardcoded API keys

6. Data Model (high-level)

Investigation / Case - id, name, created_at, target_type (domain/email/username/IP/company) Finding - id, investigation_id, module_source, data_type, value, confidence, collected_at, raw_source_reference Relationship - id, finding_id_a, finding_id_b, relationship_type (e.g., 'resolves_to', 'registered_by', 'hosted_on')

7. Phased Build Plan

Phase 1: Foundation (Weeks 1-2)

- Set up project structure, virtual environment, Docker/docker-compose skeleton - Design and implement the core data model / database schema - Build the basic FastAPI backend with a health-check endpoint - Stand up a minimal Streamlit or basic HTML dashboard shell

Phase 2: First Intelligence Module - Domain/Network (Weeks 3-4)

- Implement WHOIS + DNS record lookups - Implement subdomain enumeration via certificate transparency logs - Display results in the dashboard for a submitted domain - Persist findings to the database, not just display them live

Phase 3: Email/Username Modules (Weeks 5-6)

- Integrate a legitimate public breach-check API for email lookups - Build username enumeration across a curated list of platforms - Add async/background task handling since these lookups are slower and rate-limited

Phase 4: Correlation Engine (Weeks 7-8)

- Build logic to detect relationships between findings across modules - Add a simple relationship graph visualization (networkx + a plotting library, or a JS graph library on the frontend)

Phase 5: Reporting and Polish (Weeks 9-10)

- Build PDF/HTML export of a full investigation's findings - Add authentication if this will be exposed beyond your local network - Add rate-limit handling, caching, and error handling for flaky external APIs - Write documentation / README for the project (important for portfolio value)

Phase 6: Deployment (Week 11+)

- Containerize with Docker, deploy to a dedicated VM/LXC container on your Proxmox server - Optionally expose securely (VPN/Tailscale access only, not public internet, given the sensitive nature of OSINT tooling)

8. Portfolio Value Notes

This project demonstrates: full-stack development, API integration, async task handling, data modeling, and security/ethical awareness (a section explicitly documenting responsible-use boundaries is a strong signal to employers). It pairs naturally with your existing cybersecurity project work (IDS, AD attack/defense) - together they show both defensive (IDS) and intelligence-gathering (OSINT) skill sets, which is a compelling combination for security-focused roles. Suggested README sections when you publish this: Project overview, ethical use statement, architecture diagram, setup instructions, module list, and example (sanitized) screenshots.

9. Stretch Goals (optional, later)

- Scheduled re-scanning of saved investigations to detect changes over time (e.g., a domain's DNS records changing) - Browser extension or CLI companion tool feeding data into the same backend - Integration with your existing custom IDS project - flag if a monitored IP in your IDS logs also appears in an OSINT investigation - Threat intelligence feed integration (e.g., checking IPs against public reputation lists)